ESEP Assignment 8: Event-Driven Architecture

Nov 1, 2026 · @Joseph

Overview

You will build an AWS Lambda function that GitHub calls whenever an issue is opened on your repo, and that posts the issue link to our Slack channel.

- Event-driven architectures embody asynchronous communication between applications. Unlike applications that are self-contained and write directly to a datastore, eventing can introduce additional complexity.
- Experiencing these issues will help develop best practices and defensive programming skills.
- For submission: you will submit the link to your **public** GitHub repo. We will use this for automation to submit an issue to each repo.

The flow you are building:

1. Someone opens an issue on your GitHub repo.
2. GitHub sends a webhook (an HTTP POST with JSON) to your API Gateway URL.
3. API Gateway invokes your Lambda function with that JSON.
4. Your function reads the issue link and posts it to #eventing-webhook in your section's Slack workspace.

Grading (7 points total)

Result	Points	Submit in Canvas
Both integrations tied together: an issue on your repo posts to Slack through your Lambda	7 (full)	Link to your public GitHub repo
Only if you can't get the full project working: the GitHub webhook fires when you open an issue manually	3 (half)	Link to your Slack message

Result	Points	Submit in Canvas
The Slack URL is pushed to GitHub anywhere in your repo	−2	—

Important: if you push the Slack URL to GitHub, it is revoked for the entire class and you lose 2 points. Keep it only in the Lambda environment variable.

Part 1: Create and deploy the function

By the end of Part 1 the starter function from the template is running in AWS Lambda in us-east-2. Finish the Prerequisites at the end of this handout first.

Create your repo and project

1. Create a public repository named `esep-webhooks` (`https://github.com/{username}/esep-webhooks`), clone it, and open a terminal in it.
2. Install the Lambda project templates and the deploy tool. You only do this once per machine:

```
dotnet new install Amazon.Lambda.Templates
dotnet tool install -g Amazon.Lambda.Tools
```

If the tool is already installed, run `dotnet tool update -g Amazon.Lambda.Tools` instead.

3. Generate the function inside your repo folder:

```
dotnet new lambda.EmptyFunction --name EsepWebhook
```

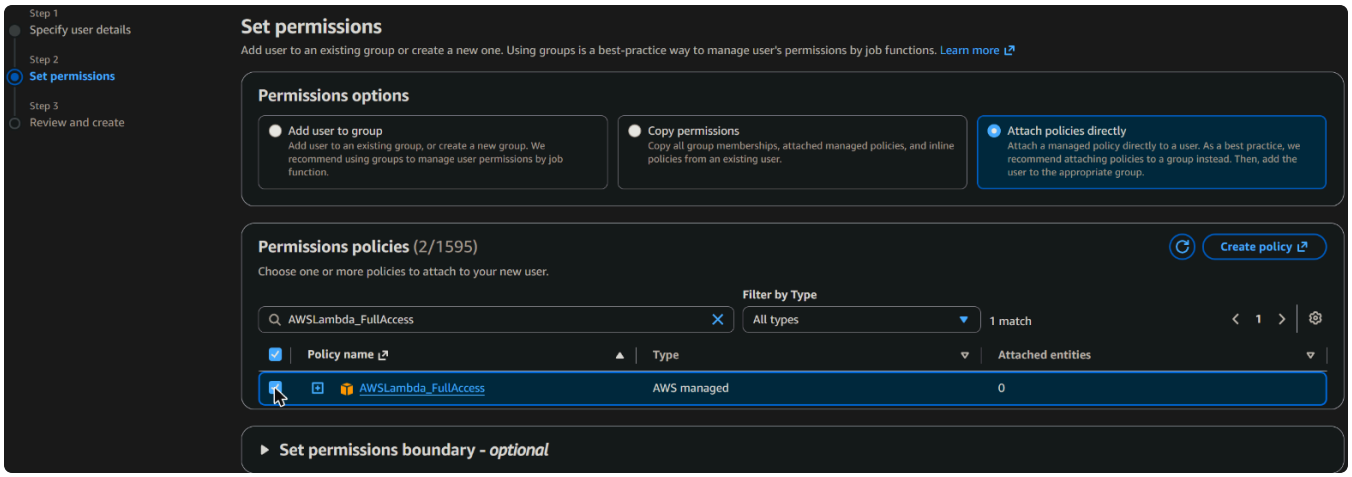
The template targets .NET 10 and the `dotnet10` Lambda runtime. Using another language? Generate a basic Lambda function with that language's CLI instead.

Set up AWS credentials

The deploy tool needs credentials, and it creates an execution role for your function, which is why the user below gets `IAMFullAccess`.

1. In the AWS console, search for **IAM** and open it.
2. Choose **Users** → **Create user**. User name: `Esep-Webhook`. Choose **Next**.

3. Under Permissions options choose **Attach policies directly**. Search for and check `IAMFullAccess` and `AWSLambda_FullAccess`. Choose **Next** → **Create user**.
4. Open the new user → **Security credentials** tab → **Create access key**. Choose **Command Line Interface (CLI)**, tick the confirmation, then **Next** → **Create access key**.
5. Choose **Download .csv file**. You cannot see the secret key again after you leave this page.
6. In your terminal run `aws configure`. Paste the two keys, set the region to `us-east-2` and the output format to `json`.



Optional: AWS now recommends `aws login` (AWS CLI 2.32.0 or newer) over long-lived access keys. It needs extra IAM setup for console sign-in, so these instructions use access keys, which work fine for this assignment.

Deploy the starter function

1. `cd` into the folder that contains `EsepWebhook.csproj`. With the current template that is `EsepWebhook`; older template versions put it in `EsepWebhook/src/EsepWebhook`.
2. Deploy:

```
dotnet lambda deploy-function EsepWebhook
```

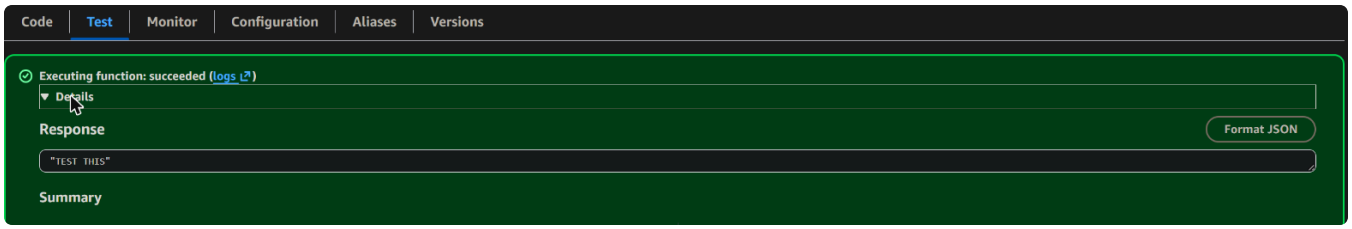
3. When asked for an IAM role, choose **Create new IAM Role**, give it a name such as `EsepWebhook-role`, and pick `AWSLambdaBasicExecutionRole` so your function can write logs.
4. Open the Lambda console, make sure the region at the top right is **US East (Ohio) us-east-2**, and confirm `EsepWebhook` is listed under **Event Functions**, the console's new name for its function list.

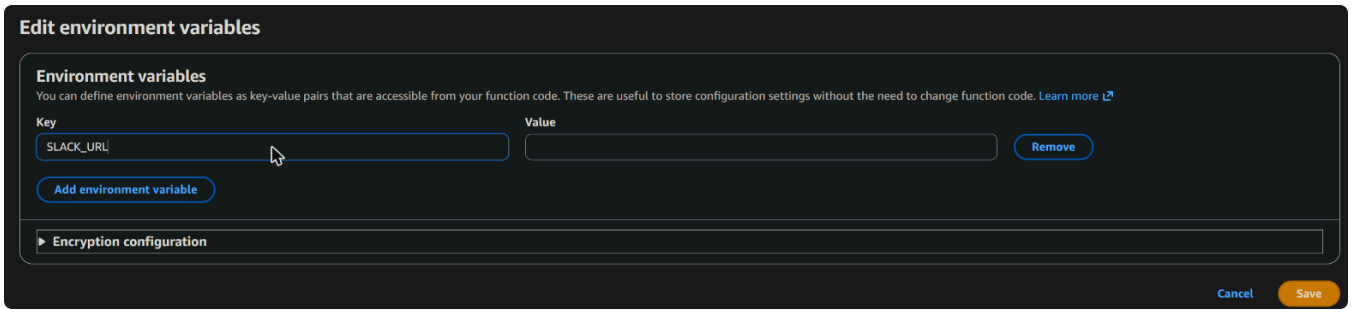


Part 2: Test the function and add the Slack URL

By the end of Part 2 the starter function runs from the console and holds the Slack URL in an environment variable.

1. In the Lambda console, open `EsepWebhook` and choose the **Test** tab.
2. Choose **Create new event** and name it `test-string`. Replace the Event JSON with a JSON string, quotes included: `"test this"`.
3. Choose **Test**. The starter function returns `"TEST THIS"`.
4. Open the **Configuration** tab → **Environment variables** → **Edit** → **Add environment variable**.
5. Key: `SLACK_URL`. Value: the webhook URL for your section's Slack workspace, provided in Canvas. Choose **Save**.





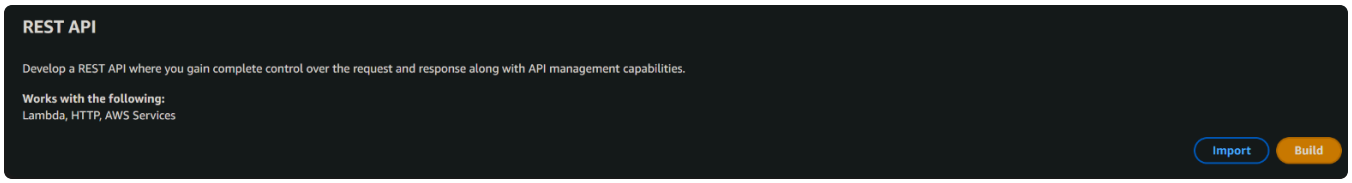
Important!! If you push the Slack URL to GitHub, it is revoked for the entire class and you lose 2 points. Keep it only in this environment variable: never in code, `aws-lambda-tools-defaults.json`, a README or a screenshot. GitHub may block a push that contains it. If that happens, remove the URL from your commit; do not bypass the block.

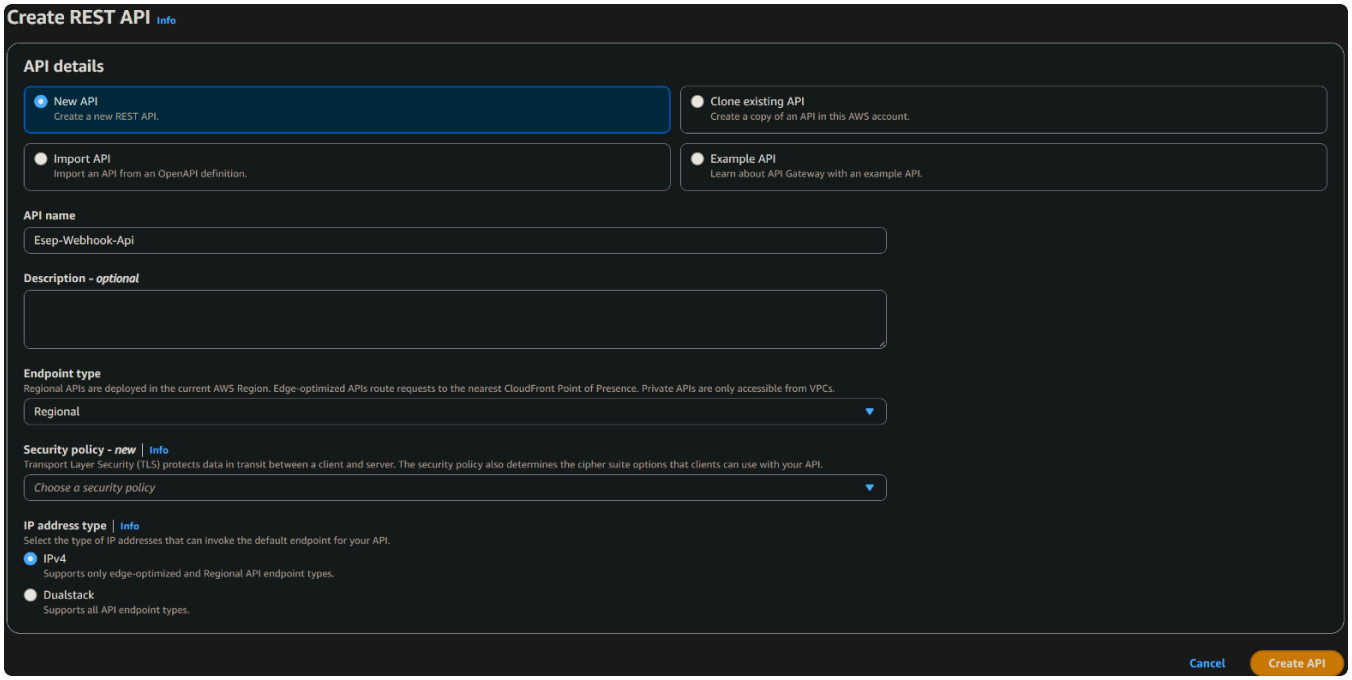
Part 3: Create the API

By the end of Part 3 you have a public URL ending in `/default` that runs your function when it receives a POST.

Create the REST API

1. In the AWS console, search for **API Gateway** and check the region is us-east-2.
2. Choose **Create API**. On the **REST API** card (not REST API Private, not HTTP API), choose **Build**.
3. Choose **New API**. API name: `Esep-Webhook-Api`. Leave the endpoint type as Regional and the other settings as they are; the new Security policy field can stay empty. Choose **Create API**.

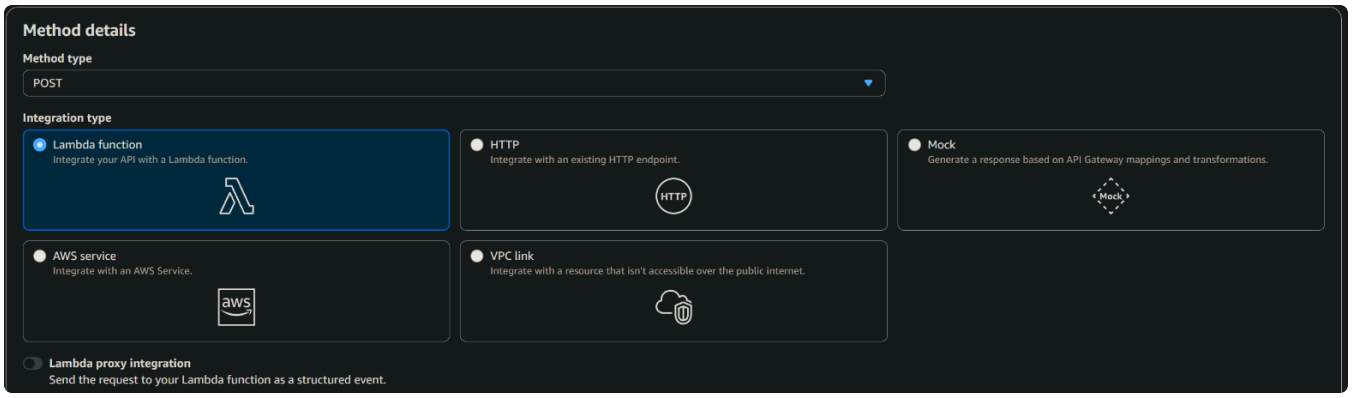




Add the POST method

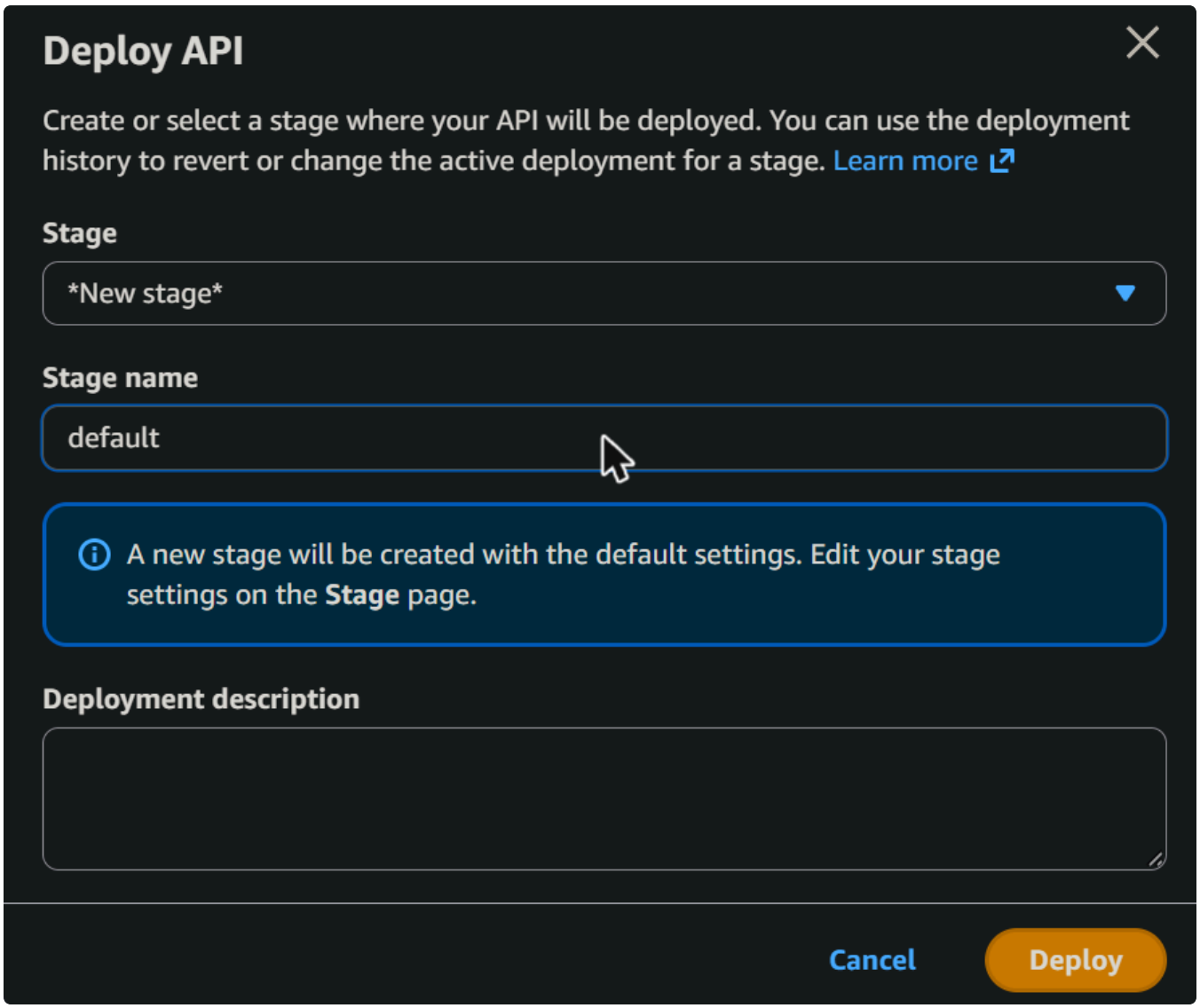
1. On the Resources page, select `/` and choose **Create method**.
2. Method type: **POST**. Integration type: **Lambda function**.
3. Leave **Lambda proxy integration** turned off. Your Part 5 code expects GitHub's JSON exactly as sent, which is what the function receives with it off.

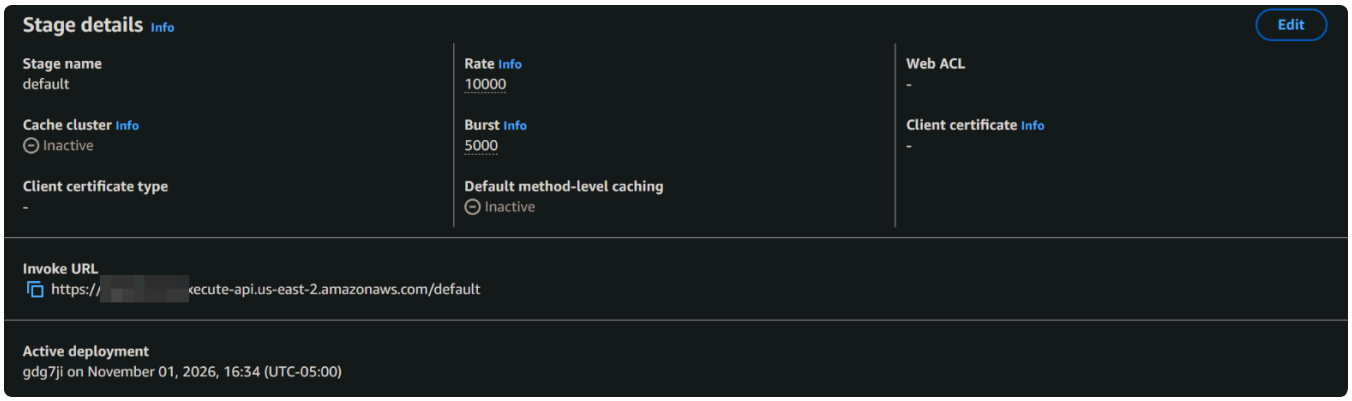
4. Under Lambda function, keep us-east-2 and choose `EsepWebhook`. Choose **Create method**. API Gateway adds the permission it needs to call your function. If a feedback survey pops up, close it.



Deploy and test

1. Choose **Deploy API**. Stage: **New stage**. Stage name: `default`. Choose **Deploy**.
2. Copy the **Invoke URL** from Stage details. It looks like `https://abc123.execute-api.us-east-2.amazonaws.com/default`.
3. In Postman, create a **POST** request to that URL exactly as copied, with nothing after `/default`.
4. Choose **Body** → **raw** → **JSON**, enter `"test"` (quotes included) and choose **Send**. You should get status 200 and `"TEST"` back.



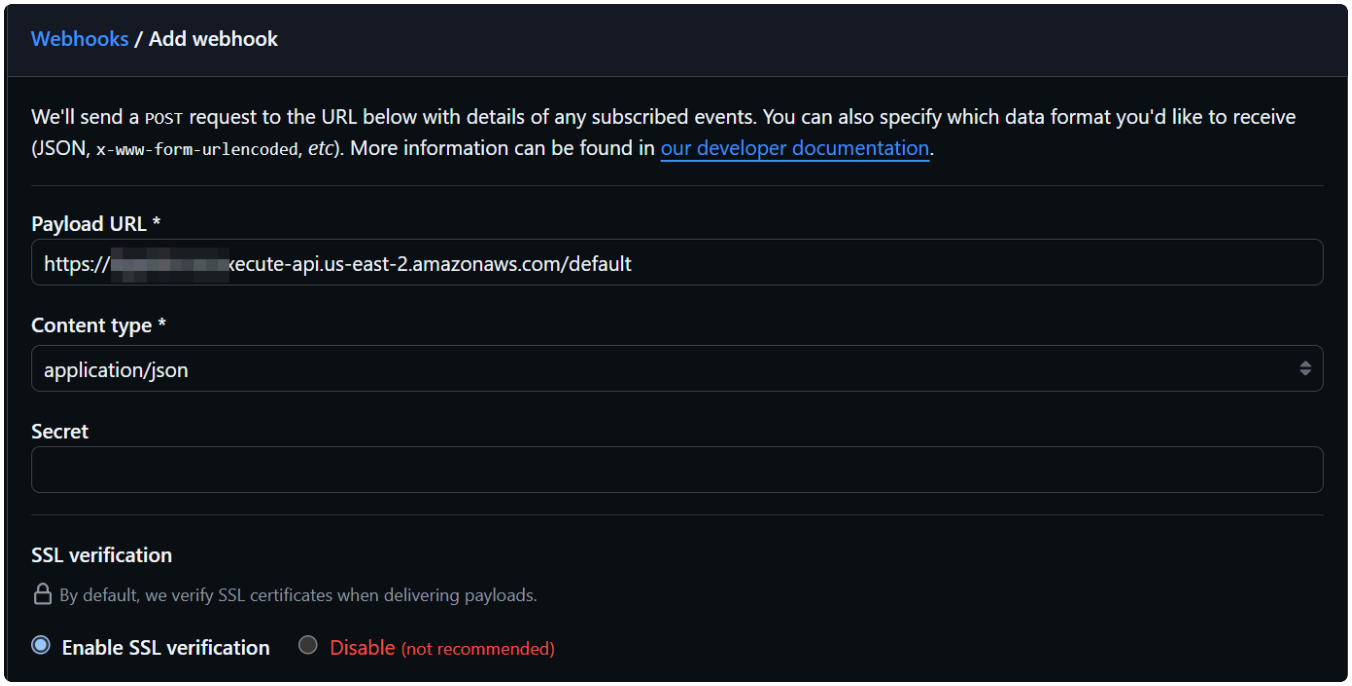


If you get 403 "Missing Authentication Token", the URL has something after `/default` or the method wasn't deployed. If you change the API later, choose **Deploy API** again and pick the existing `default` stage.

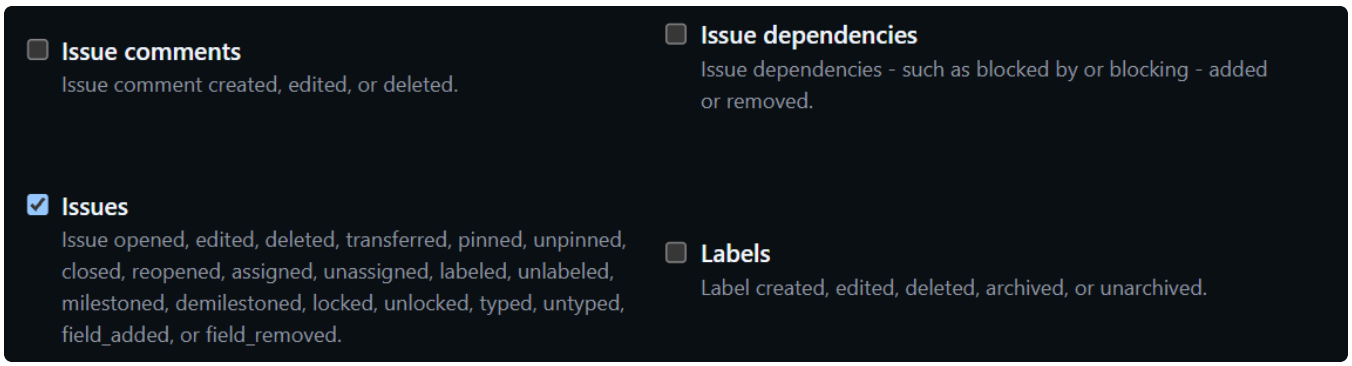
Part 4: Add the GitHub webhook

By the end of Part 4 GitHub calls your API whenever an issue changes on your repo.

1. In your repo, go to **Settings** → **Webhooks** → **Add webhook**.
2. Payload URL: your Invoke URL ending in `/default`, with nothing after it.
3. Content type: **application/json**. Your code can't read the form-encoded format.
4. Leave Secret empty and keep SSL verification enabled.
5. Under "Which events would you like to trigger this webhook?", choose **Let me select individual events**. Check **Issues** (not Related issues) and uncheck everything else, including **Pushes**.
6. Keep **Active** checked and choose **Add webhook**.







GitHub immediately sends a "ping" event. Don't worry about how the starter function answers it.

To debug later, open the webhook and choose **Recent Deliveries**. Each delivery shows the JSON GitHub sent and the response it got, and **Redeliver** sends it again. GitHub expects an answer within 10 seconds and keeps deliveries for 3 days.

Part 5: Post the issue link to Slack

By the end of Part 5, opening an issue on your repo posts "Issue Created: <link>" in #eventing-webhook.

Update the code

In the folder that contains `EsepWebhook.csproj`, add Newtonsoft.Json:

```
dotnet add package Newtonsoft.Json
```

At the top of `Function.cs`, add:

```
using System.Text;
using Newtonsoft.Json;
```

Replace the `FunctionHandler` method with this version. It reads `issue.html_url` from GitHub's JSON and posts it to the URL in `SLACK_URL`:

```
public string FunctionHandler(object input, ILambdaContext context)
{
    context.Logger.LogLine($"Received: {input}");

    dynamic json = JsonConvert.DeserializeObject<dynamic>(input.ToString());
    string issueUrl = (string)json.issue.html_url;
    string payload = JsonConvert.SerializeObject(new { text = $"Issue
Created: {issueUrl}" });

    var client = new HttpClient();
    var webRequest = new HttpRequestMessage(HttpMethod.Post,
Environment.GetEnvironmentVariable("SLACK_URL"))
    {
        Content = new StringContent(payload, Encoding.UTF8,
"application/json")
    };

    var response = client.Send(webRequest);
    using var reader = new StreamReader(response.Content.ReadAsStream());
    return reader.ReadToEnd();
}
```

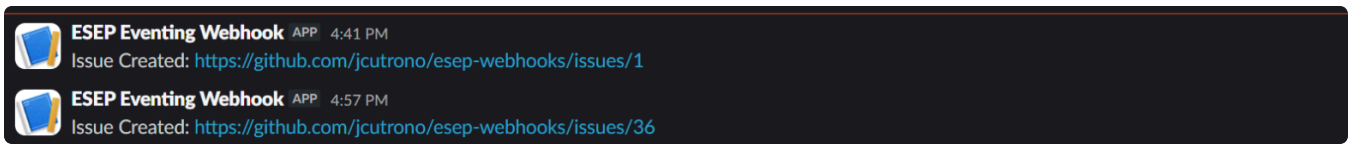
Example solution: [esep-webhooks/Function.cs].

Deploy and test

1. Run `git diff` and check the Slack URL appears nowhere, then commit and push to your public repo.
2. Redeploy with `dotnet lambda deploy-function EsepWebhook`. Afterwards, check that `SLACK_URL` is still set under **Configuration → Environment variables**.
3. Test without GitHub: in Postman, POST to your `/default` URL with this raw JSON body. A message should appear in #eventing-webhook and Postman should show `"ok"`.

```
{ "issue": { "html_url": "https://github.com/{username}/esep-
webhooks/issues/1" } }
```

4. Open an issue on your repo. Within a few seconds "Issue Created: <link>" should appear in #eventing-webhook. Editing, closing or labelling the issue also posts a message; that's expected.



If nothing shows up in Slack

- Check the webhook's **Recent Deliveries** in GitHub to see what was sent and what came back.
- Read your function's logs: Lambda → **Monitor** → **View CloudWatch logs**.
- A response of `invalid_payload`, `no_service` or `channel_not_found` comes from Slack: your `SLACK_URL` value is wrong or was revoked.

Prerequisites

Set these up before Part 1. Nothing in this class needs more than the AWS Free plan.

1. **An AWS account.** Sign up at aws.amazon.com.
   - Choose the **Free plan**. New accounts get $100 in credits, plus up to $100 more for trying certain services, and Lambda, API Gateway and IAM are all included.
   - The Free plan ends after 6 months or when the credits run out, whichever comes first. AWS then closes the account unless you upgrade to a Paid plan.
   - The standard sign-up asks for a payment method. You are not charged on the Free plan unless you upgrade.
   - If you are asked to choose a support plan, choose **Basic support (free)**.
   - If AWS offers more than one sign-up option, choose **Sign up for AWS (advanced)**. This assignment creates IAM users and roles.
2. **The .NET 10 SDK** from dotnet.microsoft.com/download. Run `dotnet --version` and check it starts with 10.
3. **AWS CLI version 2**: installation guide. Run `aws --version` and check it starts with `aws-cli/2`.
4. **A GitHub account** where you can create a public repository.

Using a language other than C#? Follow AWS's guide to packaging a Lambda function as a .zip file for your language. C# details are in AWS's C# .zip packaging guide.

Resources

- If you don't already have a preferred IDE, try VS Code.
- If you don't have a preferred way to call web APIs, use Postman.
- To see exactly what GitHub sends, open your webhook's **Recent Deliveries**, or add a second webhook pointing at webhook.site or smee.io to inspect the raw payload.